ESP32 LED Controller cURL API Integration

Command Cheat Sheet

Replace 192.168.4.1 with your device's station IP address if you are connected through your home router.

1. Querying Values

- **Query all settings & device status (JSON):**
  curl http://192.168.4.1/api/status

- **Query a single field (JSON output):**
  curl "http://192.168.4.1/api/get?var=brightness"
  curl "http://192.168.4.1/api/get?var=red"
  curl "http://192.168.4.1/api/get?var=staIP"

- **Query a single field in plain text (clean output for scripts):**
  VAL=$(curl -s "http://192.168.4.1/api/get?var=brightness&raw=1")
  echo "Current Brightness: $VAL"

- **Query Wi-Fi status:**
  curl http://192.168.4.1/api/wifi

2. Adjusting Settings (GET via URL Parameters)

- **Change brightness and colour:**
  curl "http://192.168.4.1/api/settings?brightness=180&red=255&green=0&blue=120"

- **Adjust group size and strobe interval:**
  curl "http://192.168.4.1/api/settings?group=4&interval=250"

- **Toggle strobe state (on/off flip):**
  curl "http://192.168.4.1/api/settings?enabled=toggle"

- **Pass exact RGB values (bypassing the 20-step rounding):**
  curl "http://192.168.4.1/api/settings?red=137&green=42&blue=211&raw=1"

3. Adjusting Settings (POST via Form Body)

- **Update individual or multiple parameters:**
  curl -X POST -d "brightness=220" http://192.168.4.1/api/settings
  curl -X POST -d "red=0&green=255&blue=0&interval=80" http://192.168.4.1/api/settings

4. Configuring Wi-Fi Credentials

- **Via GET:**
  curl "http://192.168.4.1/api/wifi?ssid=MyRouterSSID&password=SecretPassword"

- **Via POST:**
  curl -X POST -d "ssid=MyRouterSSID&password=SecretPassword" http://192.168.4.1/api/wifi

---

Response: Complete Device Status Inspection

curl http://192.168.4.1/api/status

This request queries the general health and state of the ESP32. When the microcontroller receives this request, the handleStatus() function serializes the entire runtime state into a JSON object and transmits it back. The payload includes the hardware identifier extracted from the MAC address, current brightness, RGB colour values, group size, strobe timing interval, strobe activation state, access point details, and station connection status including router SSID and assigned local IP. This is the primary endpoint to use if an external dashboard or monitoring system needs to sync with the physical hardware.

Querying Individual Variables in JSON

curl "http://192.168.4.1/api/get?var=brightness"

This endpoint isolates a single target variable rather than returning the entire device configuration. The var parameter is case-insensitive, meaning you can request brightness, Brightness, or BRIGHTNESS. The ESP32 parses the requested key, matches it against its active memory variables, and returns a lightweight JSON envelope containing only that variable's current value. Valid variable names include brightness, red, green, blue, group, interval, enabled (or strobe), id, apSSID, apIP, staConfigured, staConnected, staSSID, and staIP.

Querying Variables in Clean Plain Text

curl -s "http://192.168.4.1/api/get?var=interval&raw=1"

Adding raw=1 (or plain=1) alters the response header from application/json to text/plain and strips away all JSON quotes, braces, and keys. The -s flag tells curl to run in silent mode, suppressing download meters and status text. This returns exclusively the bare value, which makes it ideal for embedding directly inside shell scripts or terminal utilities without needing external JSON parsers like jq.

```
CURRENT_COLOUR=$(curl -s "http://192.168.4.1/api/get?var=red&raw=1")
echo "The active red channel intensity is: $CURRENT_COLOUR"
```

This snippet stores the exact returned number directly into a shell environment variable for conditional automation or logging routines.

Modifying Settings via GET Query Strings

curl "http://192.168.4.1/api/settings?brightness=180&interval=250"

The server route for /api/settings accepts standard GET requests with URL-encoded query parameters. This allows parameters to be modified either via curl or directly from any web browser address bar. In this example, the controller simultaneously adjusts the LED brightness to 180 and alters the flash phase duration to 250 milliseconds. The firmware handles multiple variables in a single pass, immediately recalculates internal timing, saves the new values to non-volatile flash memory via the Preferences library, redraws the LED ring, and responds with the full updated status JSON.

Modifying Settings via POST Form Data

curl -X POST -d "red=255&green=0&blue=120&group=6" http://192.168.4.1/api/settings

This command uses the HTTP POST method, which is standard for state-changing commands and prevents sensitive data from lingering in browser history or reverse-proxy access logs. The -X POST flag specifies the request method, while -d supplies URL-encoded form data in the request body. The helper function getParamValue() inspects both the request URL and the underlying POST payload, meaning the backend treats GET and POST data uniformly. This specific command sets a custom purple colour and alters the grouping so that six contiguous LEDs illuminate together, alternating with six unlit LEDs.

Toggling the Strobe Effect

curl "http://192.168.4.1/api/settings?enabled=toggle"

Rather than forcing the client to first read whether the light is on or off before sending the opposite state, the firmware contains built-in toggle logic. When the parameter enabled or strobe is given the string value toggle, the microcontroller reads its current boolean state, reverses it, writes the new state to persistent flash storage, and either blanks or lights the WS2812B LEDs immediately. Explicit states can still be enforced by passing enabled=1, enabled=0, enabled=true, or enabled=false.

Bypassing Step Quantization with Raw Mode

curl "http://192.168.4.1/api/settings?red=137&green=42&blue=211&raw=1"

The web dashboard enforces steps of 20 for RGB values and brightness to simplify UI slider behavior. By appending raw=1 to a command, the backend bypasses the rounding division math and writes the incoming integers directly into memory. This provides unconstrained 8-bit precision across all 256 levels per channel, enabling precise colour matching, colour temperature calibration, or algorithmic gradient changes driven by external scripts.

Turning Off All Output Without Changing Colour Presets

curl "http://192.168.4.1/api/settings?enabled=0"

This switches the strobe off entirely, clearing all pixels on the WS2812B ring to zero brightness without overwriting your saved red, green, blue, or interval configurations. When re-enabled later via enabled=1 or enabled=toggle, the ring immediately resumes strobing with all previous colours, grouping sizes, and timings intact.

Setting Solid Non-Strobing Light

curl "http://192.168.4.1/api/settings?interval=0&enabled=1"

Setting interval=0 instructs the rendering loop to bypass phase switching entirely. When the interval is set to zero, all LEDs determined by the group calculation remain steadily illuminated with no blinking or flashing cycles. This effectively converts the strobe fixture into a continuous task light or ring illumination lamp while retaining access to full brightness and RGB colour controls.

Inspecting Wi-Fi Connection Details

curl http://192.168.4.1/api/wifi

Sending a GET request to /api/wifi without any parameters triggers an informational response rather than a credentials update. The controller evaluates the current state of both network adapters, returning a JSON object detailing the SoftAP name, SoftAP IP, whether a home router has been configured, whether the station interface is actively connected, the home router SSID, and the dynamic IP assigned to the ESP32 by the router.

Updating Wi-Fi Credentials and Rebooting

```
curl -X POST -d "ssid=OfficeNetwork&password=SuperSecretPassword" http://192.168.4.1/api/wifi
```

This command submits new router credentials to the ESP32. The handler checks that the SSID meets the standard 1 to 32 character limit and confirms the WPA2 pre-shared key meets the minimum 8-character specification. Once validated, the new network name and password are saved to non-volatile flash memory. To prevent the connection from dropping mid-transmission, the ESP32 sends a clean HTTP 200 confirmation response back to curl first, sets an internal reboot timer for 1500 milliseconds in the future, and then issues a non-blocking hardware restart via ESP.restart() inside the primary execution loop.
